

南京大学本科期末考试试卷（究极模拟·C 卷）

—— 据最新题型情报（2 分析 + 2 改错）重排·考前终极押题卷 ——

课程名称：《高级程序设计》

学年学期：2025-2026 学年 第 1 学期

开课单位：智能软件与工程学院

考试方式：闭卷（按 C++11 标准，不考虑编译优化）

姓名：_____

学号：_____

年级：_____

专业：_____

考试时长：120 分钟；总分 100 分

题号	一	二	三	四	总分
得分					

一、简答题（共 4 题，共 20 分）

得分	
----	--

请简明扼要作答，写出定义 + 区别 / 作用 / 条件，能举简短例子更佳。

1. C++ 中 `this` 指针的作用是什么？为什么成员函数需要它？（5 分）
2. 重载单目运算符 `++` 时，前置 (`++a`) 与后置 (`a++`) 两个版本在函数签名和返回值上有什么区别？请各写出其函数声明形式。（5 分）
3. 什么情况下需要为一个类自定义析构函数？析构函数中通常需要归还哪些资源？哪些资源不需要（不应）在析构函数中归还？（5 分）
4. C++ 标准模板库 (STL) 由哪三大部分组成？其中“迭代器 (iterator)”的作用是什么？（5 分）

二、程序分析题（共 2 题，共 20 分）

得分	
----	--

请写出必要的分析过程。
（说明：返回局部对象、值传参、对象数组用临时量初始化等场景按笔试题惯例分析，即等价于编译选项 `-fno-elide-constructors`，每次相应拷贝都计入。）

1. (10 分) 阅读下面程序，回答问题。

代码	问题
<pre> #include <iostream> #include <string> using namespace std; class Grade { int score; public: Grade(int s) : score(s) { cout << "Grade(" << score << ") ctor" << endl; } Grade(const Grade& g) : score(g.score) { cout << "Grade(" << score << ") copy" << endl; } ~Grade() { cout << "Grade(" << score << ") dtor" << endl; } }; class Person { string name; public: Person(string n) : name(n) { cout << "Person(" << name << ") ctor" << endl; } Person(const Person& p) : name(p.name) { cout << "Person(" << name << ") copy" << endl; } ~Person() { cout << "Person(" << name << ") dtor" << endl; } }; class Student : public Person { Grade grade; public: Student(string n, int s) : Person(n), grade(s) { cout << "Student(" << n << ") ctor" << endl; } Student(const Student& s) : Person(s), grade(s.grade) { cout << "Student copy" << endl; } ~Student() { cout << "Student dtor" << endl; } }; Student enroll() { Student s("Cara", 88); return s; } void grade_it(Student s) { cout << "-- grading --" << endl; } int main() { cout << "===1===" << endl; Student arr[2] = { Student("Amy", 85), Student("Bob", 90) }; cout << "===2===" << endl; Student c = enroll(); cout << "===3===" << endl; grade_it(arr[0]); cout << "===4===" << endl; return 0; } </pre>	1. 写出程序的完整输出。 2. <code>Student arr[2] = { Student("Amy", 85), Student("Bob", 90) };</code> 这一句一共构造与拷贝了几个 <code>Student</code>? 请说明“临时对象 → 数组元素 → 临时对象析构”的过程。 3. <code>Student c = enroll();</code> 触发了几次 <code>Student</code> 拷贝构造? 为什么? 并写出 <code>===4===</code> 之后所有对象的析构顺序 (含 <code>arr</code>、<code>c</code>)。

2. (10 分) 阅读下面程序，回答问题。

代码	问题
<pre> #include <iostream> using namespace std; class Box { int n; public: Box() : n(0) {} Box incVal() { n++; return *this; } // 返回值 (副本) Box& incRef() { n++; return *this; } // 返回引用 void dec() { n--; } void show() { cout << "n=" << n << endl; } }; int main() { Box a; a.incVal().dec(); // (1) a.show(); Box b; b.incRef().dec(); // (2) b.show(); Box c; c.incRef().incRef().dec(); // (3) c.show(); return 0; } </pre>	1. 写出程序的完整输出。 2. 为什么 (1) <code>a.show()</code> 是 <code>n=1</code>, 而 (2) <code>b.show()</code> 是 <code>n=0</code>? 请从 <code>incVal</code> 返回值 (副本)、<code>incRef</code> 返回引用的角度, 说明 <code>.dec()</code> 究竟作用在谁身上。 3. (3) 中 <code>c.incRef().incRef().dec()</code> 执行后 <code>c</code> 的 <code>n</code> 是多少? 为什么返回引用能让链式调用真正作用在 <code>c</code> 本身?

三、程序改错题 (共 2 题, 共 20 分)

得分

下面每段程序都存在若干处错误 (语法错误或会导致运行错误的语义错误)。请逐一指出生错误的行号、错误原因, 并给出改正写法。代码左侧的数字为行号。

1. (10 分) 下面程序有 4 处错误, 请找出并改正, 简述原因。

```
1 | #include <iostream>
2 | using namespace std;
3 | class Account {
4 |     int balance;
5 |     const int id;
6 |     static int total;
7 | public:
8 |     Account(int i) {
9 |         id = i;
10 |         balance = 0;
11 |         total++;
12 |     }
13 |     static void count() {
14 |         cout << "total=" << total << endl;
15 |         balance = 0;
16 |     }
17 |     void deposit(int x) { balance += x; }
18 |     int getBalance() const {
19 |         balance += 1;
20 |         return balance;
21 |     }
22 | };
23 | int Account::total = 0;
24 | void report(const Account& a) {
25 |     a.deposit(100);
26 |     cout << "balance=" << a.getBalance() << endl;
27 | }
28 | int main() {
29 |     Account acc(1);
30 |     acc.count();
31 |     report(acc);
32 |     return 0;
33 | }
```

2. (10 分) 下面程序有 4 处错误 (含会导致运行崩溃的语义错误), 请找出并改正, 简述原因。

```
1 | #include <iostream>
2 | #include <cstring>
3 | using namespace std;
4 | class Text {
5 |     char* str;
6 | public:
7 |     Text(const char* s) {
8 |         str = new char[strlen(s) + 1];
9 |         strcpy(str, s);
10 |     }
11 |     ~Text() {
12 |         delete str;
13 |     }
14 |     Text& operator=(const Text& t) {
15 |         str = t.str;
16 |         return *this;
17 |     }
18 |     friend ostream& operator<<(ostream& os, const Text& t) {
19 |         os << t.str;
20 |     }
21 | };
22 | int main() {
23 |     Text a("hello");
24 |     Text b = a;
25 |     Text c("x");
26 |     c = a;
27 |     cout << a << endl;
28 |     return 0;
29 | }
```

四、编程题 / 代码补全题 (共 2 题, 共 40 分)

得分	
----	--

1. (20 分) 二阶矩阵类 Matrix2x2 —— 构造、加法、乘法、相等与输出。

- 根据 //TODO 注释补全 5 个成员 / 友元函数。矩阵记为 $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ 。
- 重载 + (对应元素相加)、* (矩阵乘法, 公式见注释)、== (返回 bool)、<< (friend, 格式 $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$)。该加 const 的地方不要漏。

需要补充的代码 (类定义):

```
#include <iostream>
using namespace std;

class Matrix2x2 {
private:
    double a, b, c, d; // 矩阵  $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ 
public:
    // TODO 1: 构造函数。四个元素默认 0, 支持 Matrix2x2(), Matrix2x2(1,2,3,4)。
    Matrix2x2(double a = 0, double b = 0, double c = 0, double d = 0);

    // TODO 2: 矩阵加法 (对应元素相加), 返回新矩阵。
    Matrix2x2 operator+(const Matrix2x2& m) const;

    // TODO 3: 矩阵乘法。结果  $\begin{bmatrix} a*m.a+b*m.c & a*m.b+b*m.d \\ c*m.a+d*m.c & c*m.b+d*m.d \end{bmatrix}$ 
    Matrix2x2 operator*(const Matrix2x2& m) const;

    // TODO 4: 相等比较 (四个元素都相等)。
    bool operator==(const Matrix2x2& m) const;

    // TODO 5: 输出, 格式  $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$  (friend, 支持链式输出)。
    friend ostream& operator<<(ostream& os, const Matrix2x2& m);
};
```

你的实现应能通过以下测试 (右侧注释为期望输出):

```
int main() {
    Matrix2x2 m1(1, 2, 3, 4);
    Matrix2x2 m2(5, 6, 7, 8);
    cout << m1 << endl; // [1 2; 3 4]
    cout << (m1 + m2) << endl; // [6 8; 10 12]
    cout << (m1 * m2) << endl; // [19 22; 43 50]
    cout << boolalpha << (m1 == Matrix2x2(1, 2, 3, 4)) << endl; // true
    Matrix2x2 I(1, 0, 0, 1); // 单位矩阵
    cout << (m1 * I) << endl; // [1 2; 3 4]
    return 0;
}
```

2. (20 分) 单向链表 IntList —— 头插、查找、反转与资源释放。

- 已给出 Node 结构、head 与已实现的 print(); 请根据 //TODO 补全 4 个成员函数。
- 反转要求就地完成 (不新建节点, 只改 next 指向); 析构须释放全部节点, 不得内存泄漏。
- 链表遍历牢记 `cur = cur->next;`; 反转可用 prev / cur / nxt 三指针。

需要补充的代码:

```
#include <iostream>
using namespace std;

struct Node {
    int val;
    Node* next;
    Node(int v) : val(v), next(nullptr) {}
};

class IntList {
private:
    Node* head;
public:
    IntList() : head(nullptr) {}
    IntList(const IntList&) = delete;
    IntList& operator=(const IntList&) = delete;

    // TODO 1: 析构函数——释放链表中所有节点, 不得内存泄漏。
    ~IntList();

    // TODO 2: 头插——在表头插入一个值为 v 的新节点。
    void push_front(int v);

    // TODO 3: 查找——链表中存在值为 v 的节点返回 true, 否则 false。
};
```

```

bool contains(int v) const;

// TODO 4: 反转链表——把整条链表就地反转（不新建节点，改指针指向）。
void reverse();

// 已实现: 打印, 形如 [ 3 2 1 ]
void print() const {
    cout << "[ ";
    Node* cur = head;
    while (cur != nullptr) { cout << cur->val << " "; cur = cur->next; }
    cout << "]" << endl;
}
};

```

你的实现应能通过以下测试（右侧注释为期望输出）：

```

int main() {
    IntList lst;
    lst.push_front(1);
    lst.push_front(2);
    lst.push_front(3);
    lst.print();                // [ 3 2 1 ]
    cout << boolalpha;
    cout << "has 2: " << lst.contains(2) << endl;    // true
    cout << "has 9: " << lst.contains(9) << endl;    // false
    lst.reverse();
    lst.print();                // [ 1 2 3 ]
    return 0;
}

```